Resume ATS Analyser & AI Interviewer

Project overview — architecture, data flow, scoring model and API

Generated 1 September 2026 · backend `services.py` 2,263 lines / 51 functions · model `openai/gpt-oss-120b` via Groq

1. What it does

A candidate uploads a resume PDF and pastes a job description. The app extracts a structured profile, works out what the job actually requires, scores the match out of 100 with a transparent breakdown, and can then run a typed mock interview whose questions are written from that job description.

Two layers work together throughout:

Layer	Role
DETERMINISTIC	Regex, a curated skill taxonomy and a fixed weighted formula. Same inputs always produce the same numbers. Runs with no API key and no network.
AI	Reads formatting and phrasing the regex layer cannot, and gives the final score. Every output is checked back against the source document. Optional — without a key the app still works end to end.

> The AI never gets a blank cheque. It always receives the deterministic result first and is constrained by it: it may *adjust* the score, not author it, and any entity it names that is not literally in the source is rejected in code.

2. Project structure

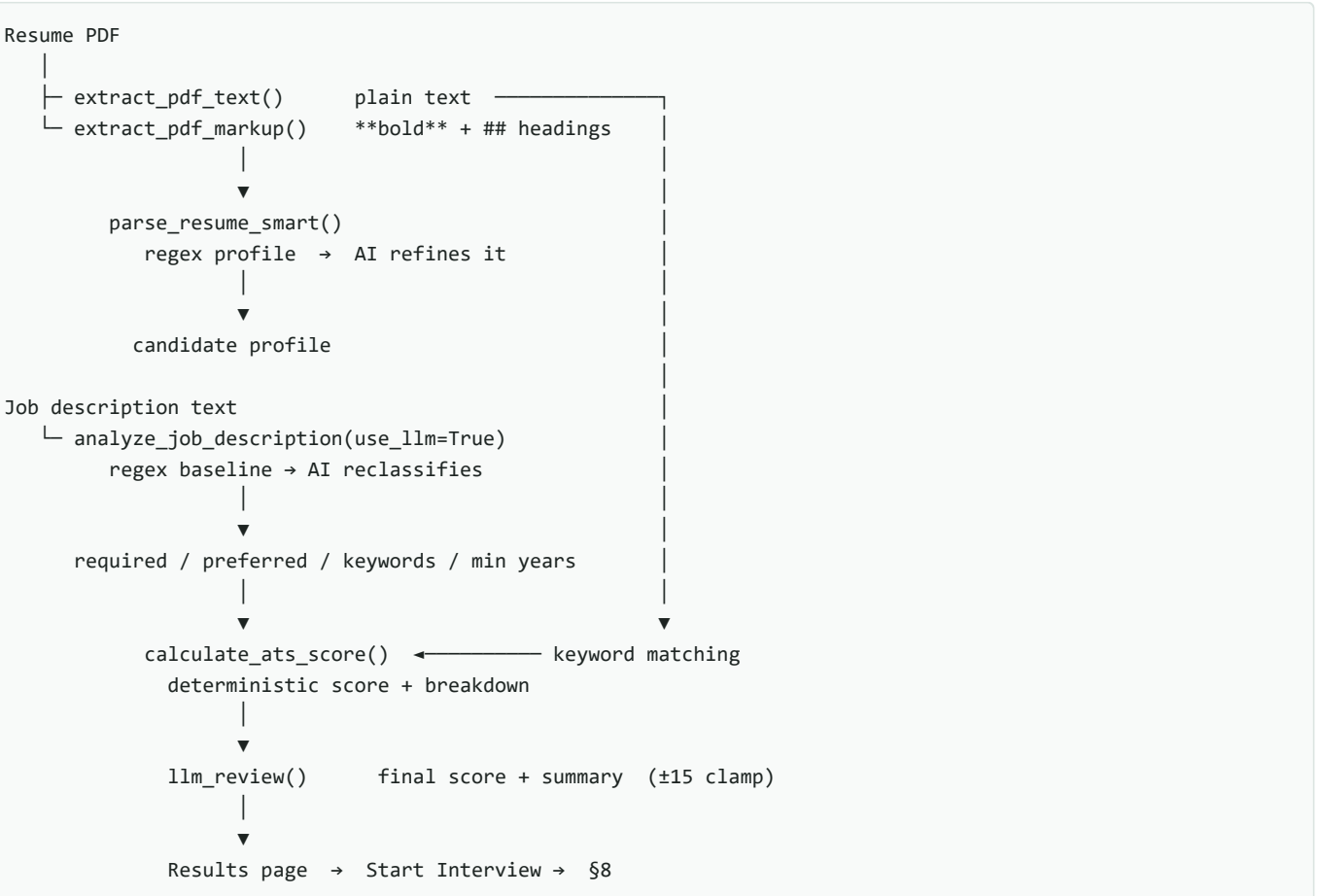
```
ai_interviewer/
├── backend/
│   ├── main.py            FastAPI app — endpoints, request models, interview sessions
│   ├── services.py        the entire pipeline (10 numbered sections, see §3)
│   └── requirements.txt   6 packages
├── frontend/
│   └── src/
│       ├── App.jsx                renders the analyser
│       ├── pages/AtsAnalyzer.jsx   inputs + which view is showing
│       ├── pages/ResultsPage.jsx   the score page
│       ├── pages/InterviewPage.jsx the mock interview
│       ├── components/ui.jsx       shared presentational components
│       ├── api/client.js           fetch wrapper
│       └── styles/global.css       design system
├── .env                   your Groq key (gitignored)
├── .env.example           template
└── README.md
```

No router — the frontend switches between three views with plain React state, so there is no `react-router` dependency.

3. Inside services.py

§	Section	What it holds
1	PDF text extraction	`extract_pdf_text` (plain) and `extract_pdf_markup` (formatting preserved)
2	Skill taxonomy	~110 canonical skills with alias regexes, plus the `SKILL_IMPLIES` table
3	Section segmentation	Splits free text on resume/JD headers — shared by both sides
4	Resume → profile	Contact, education, experience, projects, certifications, skills
5	JD → requirements	Title, required/preferred split, keywords, minimum years
6	Matching	Exact and implied skill matching, keyword matching
7	ATS score	The weighted formula and its plain-English explanation
8	One-call pipeline	`analyze()` — PDF + JD in, everything out
9	LLM review	Final score and written summary, with clamping
9a	AI JD extraction	Required vs preferred from actual phrasing
9b	AI resume extraction	Format-aware parsing from bold text and headings
10	Interviewer	Question-plan generation, grading, the progression rules

4. End-to-end flow



5. Resume extraction

Plain PDF text discards the signals that tell a reader which line is the college and which is the degree. `extract_pdf_markup` walks PyMuPDF's span tree, reads each span's bold flag and font name, measures the document's own body font size, and re-emits the page as light markdown:

```
## EDUCATION
**Gokaraju Rangaraju Institute of Engineering and Technology**
Bachelor of Technology in Computer Science and Engineering
2023 - 2027  |  CGPA: 8.9
```

The model reads those cues the way a recruiter does. Measured on a resume with real bold runs:

Field	Regex parser	Format-aware AI
Institution	*missing entirely* — captured only the degree	Gokaraju Rangaraju Institute of Engineering and Technology
Projects	**6** — description sentences became project names	**2** — the actual bold titles, with tech stacks
Experience	title was `"Acme Corp, Hyderabad"`	role / company / location split correctly
Certifications	2 entries (title and issuer line separately)	1, with issuer and date

Validation is enforced in code, not just asked for in the prompt. Every entity is checked back against the resume, comparing on letters and digits only so `**Node.js**` vs `Node . js` does not false-negative. An entity that is not there gets `verified: false`; an unverifiable *skill* is dropped outright, since skills drive the score. Soft-skill phrases and organisation-name patterns are filtered too, and every entity carries the model's `confidence`.

The regex result is the floor: skills found by either parser are kept, so enabling the AI never loses information. If the call fails, the regex profile is returned unchanged with the reason in `resume.extraction.error`.

6. Job description analysis

The deterministic pass runs first and is handed to the model as a hint. The model then reads the actual sentences, so it catches skills outside the taxonomy and judges intent from phrasing. Anything it returns that is not literally in the JD is dropped and listed in `dropped_unverifiable`.

On a real JD phrased "*It would be great if you also brought…*":

Pass	Required	Preferred
REGEX	Python, SQL, **Kubernetes, Terraform** ← wrong bucket	GraphQL
AI	Python, SQL, Apache Airflow, Snowflake, dbt	Kubernetes, Terraform, Kafka, GraphQL

The regex pass mis-filed Kubernetes and Terraform as mandatory because that phrasing is not a recognised "preferred" cue, and it missed Airflow, Snowflake, dbt and Kafka entirely — none are in the taxonomy.

Preferred skills are not mandatory. They are the extras that raise a candidate's standing when present. They carry 20% of the weight; if a JD names none, that weight is dropped and redistributed rather than counted against the candidate.

7. ATS scoring

Computed in plain Python from `services.WEIGHTS`. No model is involved in producing the number.

Component	Weight	Ratio is…
Required skills	60%	matched ÷ total required
Preferred skills	20%	matched ÷ total preferred
Experience	10%	candidate years ÷ required years, capped at 1.0
Keyword match	10%	JD keywords present in the resume ÷ total

Rescaling. If the JD never states a requirement — no preferred skills, no minimum years, no keywords — that component is marked `applicable: false` and its weight is dropped from the total, so the rest rescale to 100 instead of the score being silently capped. If the JD states nothing at all, the result is 0 and "Not scored" rather than a misleading number.

How a skill matches, both auditable and reported per skill:

- `exact` — the skill in any taxonomy spelling appears in the resume (`ML` → Machine Learning, `Python Programming` → Python)
- `implied` — a skill the candidate has stands in for it, via the hand-written `SKILL_IMPLIES` table (MySQL → SQL, FastAPI → Python, Kubernetes → Docker)

Every result carries a `summary`, an `explanation` — the arithmetic line by line, ending in the running total — and `match_detail` showing how each skill was credited.

```
Required skills — 40.0 of 60.0 points (67%): 4/6 found.
   Credited: Python, SQL, Docker, REST API. Not found: Data Structures, Algorithms.
Preferred skills — 5.0 of 20.0 points (25%): 1/4 found. Credited: AWS.
Experience      — 10.0 of 10.0 points (100%): 5.1 yrs vs 4 yrs asked for.
Keyword match   —  5.0 of 10.0 points (50%): 2/4 keywords appear.
Total: 40.0 + 5.0 + 10.0 + 5.0 = 60.0, rounded to 60%.
```

8. AI review — the final score

The whole deterministic result goes to the model: score, full breakdown, matched and missing skills, the candidate profile, and the raw resume and JD text. It returns the **final score**, a verdict, a summary, a resume summary, strengths, weaknesses and recommendations.

Two rules keep it honest:

1. It never computes the score from scratch. It sees the deterministic score and may only **adjust** it, and must explain any change.
2. The adjustment is clamped to **±** `MAX_LLM_ADJUSTMENT` (default 15 points). Out-of-band proposals are capped and reported with `clamped: true` alongside the raw `proposed_ats_score`.

The engine score stays visible next to the AI score in the UI, always. If the key is missing, the request fails, or the reply is not valid JSON, `llm_review` returns `used: false` with an `error`, the final score falls back to the deterministic one, and the app carries on.

9. The interviewer

`POST /api/interview/start` sends the job description to the model *once* and gets back a plan: 6–8 topics (JD-derived plus 2 general engineering ones), each holding a chain of three progressively deeper questions. Traversal after that is plain Python — exact and free.

Outcome	Next question
Correct, and a deeper question exists	that deeper question — **worth more points**
Correct, already at level 3	a random new topic at level 1
Incorrect	a random new topic at level 1

Points by depth: **level 1 = 10, level 2 = 20, level 3 = 30** (`services.LEVEL_POINTS`). Awarded points scale with answer quality — `points_possible × score/10` — and a wrong answer scores 0. The way to score well is to keep going deeper on one topic.

Answers are typed. Only grading costs an LLM call, one per answer; an empty answer is marked incorrect without a call. If grading fails the question is skipped unscored rather than ending the interview. `points_available` in the summary is the sum of what was actually asked, so it grows as deeper questions are unlocked.

```
Q1  Data Structures  L1  10 pts              CORRECT 9/10  →   9/10
Q2  Data Structures  L2  20 pts  « DEEPER    CORRECT 9/10  →  18/20
Q3  Data Structures  L3  30 pts  « DEEPER    CORRECT 9/10  →  27/30
Q4  SQL              L1  10 pts              WRONG   2/10  →   0/10
Q5  System Design    L1  10 pts              CORRECT 7/10  →   7/10
Q6  System Design    L2  20 pts  « DEEPER    WRONG   4/10  →   0/20
                                        61 / 110 = 55%, deepest level 3/3
```

The one piece of server state. Interview sessions live in memory in `main.py` (`INTERVIEW_SESSIONS`, capped at 200). That keeps unasked questions out of the browser, but restarting the backend ends any interview in progress. Everything else in the app is stateless — the frontend holds the resume profile and posts it back.

10. API

Endpoint	Input	Output
`POST /api/analyze-resume`	PDF, `use_llm`	`{filename, resume, resume_text}`
`POST /api/analyze-job`	`{jd_text, use_llm}`	title, required/preferred, keywords, min years
`POST /api/analyze-job/upload`	PDF/TXT	`{jd_text, job_description}`
`POST /api/calculate-ats`	`{resume, resume_text, jd_text, use_llm, jd_analysis?}`	`{job_description, ats, llm_review}`
`POST /api/analyze`	PDF + `jd_text`	everything in one call
`POST /api/interview/start`	`{jd_text, resume?, jd_analysis?, total_questions}`	`{session_id, topics, level_points, current}`
`POST /api/interview/answer`	`{session_id, answer}`	`{feedback, done, went_deeper, current}` or `{…, summary}`
`GET /api/interview/{id}`	—	progress
`POST /api/interview/{id}/end`	—	`{done, summary}` — stop early
`GET /api/health`	—	`{ok, llm_available, llm_model}`

Passing `jd_analysis` back into `/api/calculate-ats` avoids paying for a second identical LLM call and guarantees the score is computed against exactly what the user was shown.

11. Configuration and running it

```
# .env at the project root — all optional
GROQ_API_KEY=gsk_...            # without it the app is deterministic-only
GROQ_MODEL=openai/gpt-oss-120b  # llama-3.x is NOT available on this key
MAX_LLM_ADJUSTMENT=15           # how far the AI may move the score
MAX_UPLOAD_MB=10
```

List the models a key can use with `GET https://api.groq.com/openai/v1/models`. Verified working: `openai/gpt-oss-120b` (~3 s) and `qwen/qwen3.8-27b` (~2 s). `groq/compound` rejects the payload as too large.

Backend — port 8000

```
cd c:\Users\vamsh\OneDrive\Desktop\ai_interviewer\ai_interviewer\backend
..\venv\Scripts\python.exe -m uvicorn main:app --reload --port 8000
```

The `cd` into `backend` is required — uvicorn resolves `main:app` relative to the working directory, and running it from elsewhere gives "*Could not import module main*". Calling `python.exe` directly avoids PowerShell's execution-policy block on `Activate.ps1`.

Frontend — port 5173

```
cd c:\Users\vamsh\OneDrive\Desktop\ai_interviewer\ai_interviewer\frontend
npm run dev
```

Open **http://localhost:5173** — use `localhost`, not `127.0.0.1`; Vite binds IPv6 here. Vite proxies `/api` to port 8000. `.env` lives outside `backend/`, so `--reload` does not watch it: after changing a key, restart the backend manually.

12. Verification status

Area	Status
Interview progression rules	All three verified offline and live; no repeats, question cap respected, no infinite loop on a single-topic plan
LLM score clamping	10 malformed-reply cases — fenced JSON, prose-wrapped, truncated, non-JSON, non-numeric score, API exception. No case escaped the clamp or crashed
Deterministic engine	Byte-identical output across repeated runs; empty JD returns 0 / "Not scored"
API error paths	empty JD, non-PDF bytes, empty file, missing resume, unknown session, answer after finish — all correct status codes
Browser flow	Driven end to end in Chrome: upload → JD → score → interview → summary. Zero console errors
Build	`npm run build` clean; no duplicate definitions in `services.py`

13. Open items

- **Preferred-skill semantics.** Missing preferred skills currently costs part of the 20% weight. An alternative reading is a pure bonus — present adds, absent never subtracts. This is a one-function change in `calculate_ats_score` and is still an open decision.

- **Scanned resumes.** Image-only PDFs are rejected with a clear message rather than OCR'd. Adding OCR would mean a Tesseract dependency.
- **Interview durability.** Sessions are in memory; a backend restart ends an interview in progress. Persisting them needs a store.
- **Not built.** Voice, avatar and video were removed from scope and are not present anywhere in the codebase.